Laborator nr. 1

1. Introducere în sistemul de operare Linux

Scurt istoric:

Sistemul de operare UNIX a fost creat la Bell Laboratories in 1969, de catre Ken Thompson. În 1971 UNIX-ul este rescris împreuna cu Dennis Ritchie în C, devenind multi-tasking. Portabilitatea mare a programelor scrise în C a condus la o răspândire destul de rapidă a limbajului C și a sistemului de operare UNIX. De-a lungul anilor au existat numeroase versiuni, realizate de diferite companii sau universități. S-au impus doi mari contributori:

- compania AT&T și Bell Laboratories – au dezvoltat versiunile succesive de SYSTEM V UNIX
- Universitatea Berkeley – au dezvoltat versiunile succesive de BSD UNIX

În 1991, Linus Torvalds a creat un sistem de operare din familia UNIX care rula pe sistemele compatibile IBM-PC. Sursele au fost făcute publice pe internet, și o mulțime de oameni au început sa-i raspunda, sa-i aduca imbunatatiri, noi sugestii, etc.

Deși a fost rescris de la zero, Linux se bazează în întregime pe comenzile și „look and feel”-ul UNIX. Fiind un un sistem de operare multiuser si multitasking, Linux este folosit cu precădere pe servere și începe sa ocupe un loc important si pe sistemele desktop.

Linux este sub licenta GPL ceea ce înseamnă următoarele:

- este disponibil in cod sursa gratuit.
- oricine vrea sa aducă modificări, sau sa folosească anumite parti este liber să o facă, dar produsul nou obtinut sa fie sub aceeași licență (adică să fie gratuit și cu sursele la vedere).

De unde se poate obține Linux ? Cat costa ? Ce aplicații sunt disponibile?

Exista mai multe **distribuții** Linux: Slakware, RedHat, Fedora, CentOS, AlmaLinux, Rock Linux, Debian, Ubuntu, Knoppix, SuSe, OpenSuse, Gentoo, ...

O distributie reprezinta nucleul Linux impreuna cu anumite fișiere de configurare și programe. Ele nu diferă decât prin versiunea nucleului inclus, aplicațiile (numărul și versiunile incluse), modul de instalare, configurare si întreținere. Distributiile de Linux pot fi descarcate gratuit de pe Internet. În cazul distribuțiilor comerciale precum RedHat sau Suse se cumpara doar suportul tehnic sau aplicațiile proprietare care le însoțesc, sursele principale fiind disponibile.

Programe disponibile sub Linux:

Majoritatea distributiilor Linux includ compilatoare pentru diverse limbaje de programare, utilitare pentru rețea (email, telnet, ftp, www), creare și manipulare documente, tipărire, arhivare, si multe altele. Pe langa acestea exista programe disponibile pe Internet, unele gratuite, altele nu, iar marile firme de software au început sa porteze aplicațiile lor pe Linux.

Iată cateva exemple:

OpenOffice/LibreOffice, KOffice (foarte asemănătoare cu MS-Office), Gnumeric (calcul tabelar), AbiWord (editor de texte), Kpdf, Ggv, xpdf, okular (viewer documente pdf)

Mozilla, Konqueror, Opera, Chrome (browsere Web)
Thunderbird, Evolution (client de mail asemanator cu Outlook)
xmms, mplayer – aplicații multimedia
blender – grafica 3D
K3B, xcdroast – soft pentru inscriptionare cd/dvd
Amanda, bacula, backuppc - sisteme de backup
Systemrescue, partimage, clonezilla, g4l – utilitare echivalente NortonGhost
Ejabberd, wildfire, pidgin, skype – instant message (servere si clienți)
Octave, Scilab (asemănătoare cu Matlab), Matlab, Mathematica, Maple (calcul simbolic si nu numai)
KDeveloper (asemanator cu Visual Studio), CodeBlocks, Anjuta, QTDesigner, IntelliJ, JBuilder, Eclipse, NetBeans – medii de dezvoltare
RCS, CVS, Subversion, Perforce, git – servere de management al versiunilor
MySQL, PostgreSQL, Oracle, DB2, Informix - servere baze de date
OpenLDAP, Fedora Directory Server – server LDAP
Jboss, Tomcat, Glassfish - servere de aplicații java
Apache, nginx,node.js – servere web
Sendmail, postfix, qmail – servere de mail
Proftp, vsftp – servere FTP
Samba – server de fișiere pentru stații windows (poate oferi si servicii de autentificare înlocuind cu succes un Windows Server controler de domeniu)

Informații utile: www.linux.org, www.redhat.com, www.debian.org, www.sourceforge.net, www.osalt.com, alternativeto.net

2. Instalarea sistemului Linux

Încă de la început, Linux-ul a avut nevoie de resurse hardware minime pentru a rula. Acest lucru nu s-a schimbat în timp. Linux-ul poate functiona pe sisteme cu resurse hardware foarte limitate însă a fi capabil sa rulezi Linux-ul nu este totuna cu a crea un sistem server.

Evident, cu cat este mai bun hardware-ul cu atat performanța obținută la rularea sistemului de operare va fi mai mare. Cu toate acestea ultimele descoperiri în domeniul hardware nu sunt suportate de Linux. De exemplu, dacă o placă video cu ultimele facilități a apărut pe piața nu va așteptați ca o anumită distribuție de Linux să o și recunoască, deși marii producători precum Nvidia si AMD oferă suport si drivere. Producătorii specifica pe ce versiuni de Linux se poate instala placa. Uneori, producătorii hardware îngreunează accesul dezvoltatorilor Linux la informațiile specifice necesare pentru a scrie driverele si astfel devine dificil ca aceștia sa implementeze rapid suportul pentru ultimele produse hardware. Pentru unele plăci wireless producătorii nu oferă drivere si suport pentru Linux. Acest lucru a dus la apariția de aplicații/proiecte specializate precum *madwifi* pentru placile cu chipset Atheros sau *ndiswrapper* care foloseste informatiile din driverele de windows pentru a se putea folosi plăcile wireless.

2.1. Pregatirea instalării

RedHat Linux, distribuția Linux la care se va face referire în continuare, se poate instala prin intermediul serviciilor FTP, NFS sau SMB (Samba) atunci cand CD-ROM-ul sau DVD-ul Red Hat se afla pe un alt computer. Aceste metode sunt foarte sensibile si de puține ori funcționează fără erori. Pentru a economisi timp si efort este recomandabil sa realizati instalarea folosind unitatea CD/DVD de pe calculatorul pe care instalați Linux-ul.

Deoarece aceasta este metoda recomandata, vom considera în continuare să faceți instalarea de pe CD/DVD-ul propriu. Este posibila si instalarea folosind bootarea din rețea, dacă placa de retea si bios-ul de pe placa de baza suporta acest lucru.

2.2. Instalarea propriu-zisa

Pașii care vor fi urmati la instalarea RedHat Linux pot fi urmati si în cazul distribuțiilor Fedora, CentOS, Scientific Linux deoarece sunt distribuții înrudite sau derivate si perfect compatibile cu acesta, singura diferenta fiind pachetele de programe incluse în distribuție. RedHat Linux, fiind o distributie comerciala include si pachete software proprietare ce nu sunt disponibile la nivel de cod sursa si prin urmare nu sunt introduse în distribuțiile derivate. Spațiul pe disc necesar pentru instalarea RedHat Linux sau a oricărei distribuții derivate variază de la ~475MB la 6-7GB în funcție de numărul de aplicații pe care dorim sa le instalam.

Deoarece procesul de instalare are o interfata grafica ușor de utilizat si cu multe explicații vom insista asupra unui aspect mai important: partitionarea discului.

Imediat după bootarea sistemului de pe CD/DVD, pe ecran se va observa un mesaj de intampinare si un prompter:

`boot:`

Apăsați tasta Enter pentru a porni o instalare/upgrade normal.

2.2.1.Partitionarea discurilor

Partitionarea discurilor este un pas foarte important deoarece modificarea ulterioara a partitiilor nu ofera foarte multa siguranta in fata posibilitatii pierderii de date. De aceea fiți foarte atenți la acest pas al instalării.

Dacă aveți partiții ale unui alt sistem de operare instalat anterior (de ex. Windows) acum puteți șterge aceste partiții sau puteți folosi partițiile nefolosite pentru a crea un sistem cu încărcare dublă: Linux Red Hat și Windows.

În continuare se presupune ca aveti numai partitii Linux (situație existentă atunci cand realizati o instalație tip Server).

Red Hat va permite sa va partitionati fiecare disc, unde fiecare partiție este notată utilizand o schema de genul `hd[a-z][N]` pentru discurile hard IDE si o schema `sd[a-z][N]` pentru discurile hard SCSI. Aici N este numărul partiției. De exemplu, *hda1* este partiția numărul 1 de pe primul disc IDE, *`hda`*, si, similar, *`sda1`* este prima partitie de pe primul disc SCSI/SATA. Va trebui să vă decideți singuri cum va veti partiționa discurile. În continuare sunt prezentate cateva sfaturi.

Aveți nevoie de o partitie */* **(root)** pentru a stoca kernelul și fișierele legate de acesta. Aceasta partiție nu trebuie sa fie foarte mare. Însă, dacă dispuneți de spațiu, încercați sa o faceti de suficient de mare. Asigurati-va că ați ales pentru aceasta partiție tipul Linux native (sau ext2/ext3/ext4/xfs, în funcție de versiunea de Red Hat pe care o instalație.

Tot obligatoriu trebuie sa creati o partitie **swap** (tipul partitiei: swap) care sa aiba o dimensiune egala cu de doua ori marimea memoriei RAM (recomandari pot fi gasite la adresa https://access.redhat.com/documentation/en-US/Red_Hat_Enterprise_Linux/6/html/Storage_Administration_Guide/ch-swapspace.html#tb-recommended-system-swap-space).

Opțional, puteți crea partiile:

- **/usr** (tipul partiției trebuie sa fie Linux native/ext2/ext3; este o partitie care trebuie sa fie cat mai mare),
- **/home** pentru directoarele rădăcina ale utilizatorilor mai puțin utilizatorul root (tipul partiției trebuie sa fie Linux native ext2/ext3/ext4/xfs; dacă aveți mulți utilizatori trebuie sa va decideți care este spațiul pe care îl alocați pe utilizator, să multiplicați aceasta valoare cu numărul maxim de utilizatori pe care considerați ca îl veți avea si să creați o partitie corespunzătoare ca dimensiune),
- **/boot**: uneori este recomandata crearea unei partitie de 1GB pentru stocarea imaginii de boot a sistemului.
- **/var** pentru cozile si log-urile sistemului.

În cazul în care avem nevoie sa instalam Linux pe mai multe calculatoare identice din punct de vedere hardware, putem automatiza procesul de instalare folosind atat aplicații incluse în distribuție (**kickstart file**), fie aplicații specializate: **systemimager**.

Aceste utilitare permit automatizarea completa a instalării, inclusiv partitionare, creare conturi utilizatori, setare parole, pornire servicii sistem dupa instalare.

3. Componentele sistemului de operare Linux

Sistemul de operare Linux foloseste un sitem de procese numit multitasking pentru a aloca timpul de lucru al procesorului mai multor utilizatori în același timp. În realitate nu este vorba de procese care se rulează în același timp, ci unele după altele la intervale de timp de ordinul milisecundelor.

Sistemul de operare poate aloca timpul în așa fel incat sa lase impresia de continuitate pentru fiecare utilizator. În timp ce unul dintre aceștia lucrează la terminalul sau el împarte cu ceilalți resursele de calcul.

Linux, ca oricare alt sistem de operare, este un set de programe care administrează toate operațiile sistemului de calcul. El asigura de asemenea o interfata intre utilizator si resursele sistemului convertind cererile care vin de la mouse sau tastatura în operații de efectuat în sistemul de calcul.

Majoritatea sistemelor de operare, inclusiv Linux, au trei mari componente: Kernel-ul, Shell-ul si Sistemul de Fișiere. **Kernel**-ul administrează operațiile computerului. **Shell**-ul asigura o interfata pentru interacțiunea dintre utilizator si calculator iar **Sistemul de Fișiere** asigura un mijloc de organizare si gestionare a informațiilor pe discurile hard ale calculatorului.

Kernel-ul este “inima” sistemului de operare si oferă mijloacele primare necesare ca un sistem de calcul sa functioneze. Kernel-ul se afla cel mai aproape de CPU si hardware. Este un fișier executabil care se încarcă atunci cand porneste calculatorul. Odată ce kernel-ul a fost încărcat el realizează următoarele funcții:

- Gestionează dispozitivele, memoria, si procesele.
- Controlează interacțiunea (transmisia informației) dintre programele aplicație (utilitati) si hardware-ul sistemului.
- Administrează funcții precum: spațiul swap, sistemele de fișiere

Un shell este o interfata intre utilizator si kernel. Acționează ca un interpretor sau ca un translator. Cu alte cuvinte, shell-ul accepta comenzi scrise de utilizator, le interpretează, si apoi executa programele corespunzătoare. Shell-urile pot fi linie de comanda sau grafice.

Shell-ul este deci legatura directă, interfata directă a utilizatorului cu sistemul de operare. De obicei, cele mai multe cereri sunt pentru a rula programe.

Sistemul de fișiere asigura o modalitate de a separa si de a organiza informația pe

un hard disk. Structura de fișiere a sistemului Linux este o ierarhie de directoare, subdirectoare si fișiere care sunt grupate în funcție de anumite scopuri. Sistemul de fișiere este integrat cu Kernel-ul în scopul asigurarii unei structuri organizatorice pentru software-ul si pentru datele utilizatorilor.

4. Sistemul de fișiere Linux/Unix

Sistemul de fișiere este o caracteristică importantă a unui sistem de operare. Sistemul de fișiere este locul unde se memoreaza pe suport fizic fișierele sistem si cele utilizator. Sistemul de fișiere este caracterizat prin trei aspecte: structura ierarhică, independența ridicată față de hardware si o mare flexibilitate. Structura ierarhică este organizată sub forma unui arbore cu un director radacina (root). Directorul rădăcină (/)poate conține fișiere, legături sau alte directoare, numite subdirectoare. Subdirectoarele sunt noduri in arbore, iar fișierele sunt frunze. Independența față de hardware rezulta din faptul ca fișierele sunt privite ca o succesiune de octeți. Flexibilitatea se bazează pe posibilitatea de a monta sau demonta la orice nivel în ierarhia de fișiere noi structuri arborescente de directoare si fișiere.

Nivelul de sub rădăcina este tipic, fiind comun tuturor sistemelor de operare compatibile UNIX. Caracteristic pentru toate este faptul ca pe masura ce ne departam de radacina diferentele intre sisteme cresc. Administratorul de sistem stabilește de regula structura sistemului de fișiere si tine cont de resursele hard de care dispune si de cerințele utilizatorilor. Structura standard contine directoarele:

/bin	Comenzi necesare în timpul procesului de bootare si care ar putea fi folosite de utilizatorii obișnuiți (probabil după bootare).
/sbin	Ca si /bin, însă comenzile nu sunt pentru utilizatori obișnuiți, deși acestia le-ar putea folosi la nevoie dacă li se permite.
/etc	Fișiere de configurare specifice.
/root	Directorul propriu al utilizatorului radacina (root).
/lib	Bibliotecile necesare programelor din sistemul de fișiere root.
/lib/modules	Module kernel incarcabile, în special acelea care sunt necesare pentru a boota atunci cand se incearca recuperarea dupa dezastre.
/dev	Fișiere dispozitiv.
/tmp	Fișiere temporare. Programele care rulează după incarcare (bootare) ar trebui sa utilizeze /var/tmp si nu /tmp, atat timp cat primul se afla probabil pe un disc cu mai mult spațiu.
/boot	Fișiere utilizate de bootstrap loader, de exemplu GRUB, LILO. Imaginile Kernel sunt ținute adesea aici în loc de directorul rădăcină. Dacă exista mai multe imagini ale kernel-ului, directorul poate deveni foarte mare, de aceea este de recomandat sa fie ținut într-un sistem de fișiere separat.
/mnt	Puncte de montare pentru montari temporare făcute de administratorul de sistem. /mnt ar trebui sa fie împărțit în subdirectoare (de exemplu, /mnt/dosa ar fi partiția ce utilizează un sistem de fisiere MS-DOS, iar /mnt/exta ar fi același cu un sistem de fisiere ext2).
/usr	Conține comenzi executabile, utilitati diverse si diferite biblioteci..
/var	Conține log-uri, mesaje de eroare, o serie întreaga de utilitare folosite pentru imprimare, sistemul de posta electronica, DNS, etc.
/home	Conține directoarele utilizatorilor.

Structura arborescentă a sistemului de fișiere da posibilitatea unui număr mare de utilizatori sa creeze si sa gestioneze un număr mare de fișiere. Nu exista limitare a

numărului de noduri eventualele restricții sunt impuse de hardware. Particularitati:

* în directoare diferite pot exista fișiere diferite cu același nume;
* un fișier frunza poate fi legat la mai multe directoare chiar dacă acesta nu se afla la același nivel in arbore;
* fiecare director contine cel putin doua intrari:
  . referă directorul însuși, iar
  . . referă directorul superior (părinte);
* orice director este legat la nivelul superior printr-o singura legatura (..).

Accesul la un fișier se face prin indicarea poziției lui în arbore. Pentru aceasta se indica modul de obținere al fișierului coborand pe nivelurile arborelui fie de la directorul curent (cale relativa) fie de la rădăcina (cale completa). Prin directorul curent se înțelege directorul în care se găsește utilizatorul la momentul curent. Numele complet al fișierului cu indicarea poziției sale în arbore poarta denumirea de nume de cale. Numele de cale poate fi de orice lungime, fiecare nume de director fiind separat de vecinul sau prin caracterul '/'. Caracterul '/' pe langa rolul de separator de directoare si fișiere identifica si directorul rădăcină. Pentru utilizatorii familiarizați cu sistemul de operare Windows sunt utile observațiile:

- caracterul '\' este înlocuit in Unix cu '/'.
- în Unix se face distincție între litere mici si litere mari;
- numele unui fisier Unix poate fi oricat de lung în funcție de implementare (minim 14 octeți);
- extensia nu este o caracteristică a sistemului Unix;
- un fișier Unix poate avea mai multe nume, fiecare fiind asociat unei legături;
- un fișier executabil în Unix este un fișier ce are setat dreptul de execuție (indiferent de numele sau);
- un fișier de comenzi Unix se numește script.
- se acceptă caracterele * si ? în Unix, dar semnificația diferă;
- drepturile de acces asupra unui fisier in Unix sunt diferite. De exemplu, un fisier ascuns în Unix înseamnă lipsa dreptului de citire pentru directorul care conține fișierul.
- un fișier text Unix are marcat sfârșitul de linie printr-un singur caracter ('\n');
- pipe-ul în Unix nu impune crearea de fișiere temporare;

4.1. Tipuri de fișiere

În Unix se deosebesc patru tipuri de fișiere: ordinare (obișnuite), pipe, speciale si directoare. Unele documentații considera fișierele **pipe** în categoria fișierelor speciale.

Din punct de vedere al sistemului de operare un fișier este un șir de octeți de lungime nedefinita terminat cu un caracter special de sfarsit de fișier (**EOF**). Orice octet sau secvența de octeți poate fi citită sau scrisă. Citirea sau scrierea începe de la o locație de octet specificata prin pointerul de fișier (sau marker), care poate fi pozitionat oriunde în fișier. Fișierele obișnuite sunt păstrate pe disc.

Un fișier obișnuit este creat de un proces. El poate contine o sursa (text) sau un fișier executabil (binar). Două sau mai multe procese pot sa citească și să scrie concurent în același fișier. Rezultatul depinde de ordinea în care cererile individuale de I/O apar si sunt în general imprevizibile. Până nu demult Unix-ul nu avea mecanisme eficiente pentru controlul accesului concurent. Versiunile mai noi de Unix dețin un control al concurenței prin semafoare.

Fișierele obișnuite nu au nume, ele au atașate niște numere, care sunt interpretate ca index intr-o lista de **i-node**-uri, pastrata la inceputul fiecarui disc care contine un sistem de fișiere Unix. Fiecare i-node conține informații importante despre fișier, care nu includ însă numele fișierului sau octeții de date. Asupra informațiilor din i-node se va reveni pe

parcurs.

Un fișier **pipe** este un fișier care este citit de un proces o singura data si este de natura temporara. Dacă data a fost citită din pipe o citire ulterioară este posibila doar dacă procesul care a creat fișierul pipe recreeaza datele într-un nou fișier pipe. Fișierele pipe sunt cunoscute ca fișiere FIFO. Asupra acestor fișiere se va reveni în lucrările următoare.

Fișiere speciale sunt fișiere atașate dispozitivelor de I/O (driverele bloc sau driverele caracter). În directorul `/dev` se găsesc toate referințele la dispozitivele de I/O: discuri, benzi magnetice, terminale, imprimante, mouse etc. Acestea sunt considerate fișiere speciale. De exemplu, pentru fiecare partiție a unui hard disc se găsește cate un fișier special. Un fisier special deține un i-node, care însă nu referă un bloc de date pe disc. In schimb acest i-node conține un număr de dispozitiv, care este folosit ca index într-o tabela kernel de proceduri pentru dispozitive periferice. Pentru identificarea fiecărui dispozitiv se folosesc doua numere: minor (identifica numărul dispozitivului de tipul dat) si major (identifica tipul dispozitivului).

Folosirea dispozitivelor in aceasta maniera confera avantajul tratării uniforme. Din punct de vedere utilizator nu exista nici o diferenta intre lucrul cu fisiere ordinare si cele speciale. De exemplu:

```shell
$ cp prg.c /usr/acct/k/prg1.c # copiere simplă
```

Fișierele speciale de tip bloc respecta cerința: pe de o parte dispozitivul conține un șir de blocuri de dimensiune fixa (în mod uzual de 512 octeți fiecare), iar pe de cealalta parte buffer-ele kernel-ului sunt utilizate ca buffer-e cache pentru rapidizarea operației de I/O. Fișierele speciale de tip caracter nu respectă nicio cerinta. În acest caz, operațiile de I/O se fac fie la nivel de octet fie la nivel de piste.

Un director face legătura între numele fișierelor si locul unde acestea sunt memorate pe disc. El nu contine efectiv fișierele care îi aparțin ci doar referințele la acestea, sub forma unei succesiuni neordonate de intrări de 16 biți (sistemele noi de Unix nu mai au aceasta limitare). Orice intrare director este formată din numele fișierului (14 octeți) si un număr de identificare (2 octeți). Acesta din urma referă un i-node sau nod de identificare, în care se păstrează informațiile referitoare la fișierul în cauză. Fișierul director conține o tabela de nume de fișiere împreună cu inode-urile asociate.

Fiecare fișier are un singur i-node care contine:

1. Proprietarul fișierului indicand si drepturile de acces ale celorlalți utilizatori. Super User-ul ignora drepturile indicate în acest camp.
2. Tipul fișierului (obișnuit, director, pipe sau special)
3. Drepturile de acces la nivel de utilizator, grup si alții.
4. Timpul ultimului acces si ultimei modificari, data si ora ultimei modificari efectuate asupra i-node-ului.
5. Numărul de legături (a se vedea comanda unlink).
6. Adresele disc ale sectoarelor ce conțin datele fișierului.
7. Lungimea fișierului în octeți.

Dacă i-node-ul este 0 intrarea respectivă este vidă, adică nu există fișiere în acel director.

Un director poate fi citit ca orice fișier, dar nu poate fi scris în mod direct, ci numai prin intermediul anumitor programe.

Conversia numelui de cale specificata in programe (procese) în i-node-ul fișierului este sarcina kernel-ului Unix.

O caracteristică importantă a sistemului este noțiunea de legare (linking). În acest caz vom avea doua nume de fișiere în doua directoare diferite care referă același i-node. Pentru legarea la orice director a unui fișier care este legat deja la un director sistemul pune la dispoziție comanda ln (link). Sintaxa: `ln nume_cale_veche nume_cale_noua`

Același lucru se poate realiza prin apelul sistem **link**. Ștergerea unei legături se poate realiza prin comanda sau apelul sistem **unlink**. La ștergerea unei legături dintr-un director i-node-ul si blocul de date asociat nu este disponibilizat căci ar putea sa mai existe un alt fisier care sa refere același i-node. Din acest motiv i-node-ul conține în structura sa un camp contor de legături. Ștergerea unei legături implica decrementarea contorului de legături. Cand acesta ajunge la zero kernel-ul eliberează i-node-ul si blocul de date. **Se recomanda insa atenție mare la folosirea acestor comenzi sau apeluri sistem**.

O alta caracteristică importantă a sistemului Unix este posibilitatea de a monta un întreg arbore de fișiere undeva în ierarhia altui sistem de fișiere. Nucleul sistemului de operare Unix recunoaște un singur director radacina. Dar este posibil ca fisierele sa se găsească pe mai multe suporturi fizice sau logice, fiecare volum avand un sistem de fișiere propriu arborescent. Este posibila suprapunerea rădăcinii unui astfel de sistem de fișiere pe un director al sistemului de fișiere recunoscut de nucleu. Aceasta operație se numește montare (mount) si poate fi realizata de superuser. Înaintea acestei operații este necesar un punct de montare, care poate fi orice director gol din sistem.

După montare, calea la punctul de montare prefixeaza orice acces la un fișier sau director de pe sistemul de fișiere montat. De exemplu:

```shell
$ mount /dev/disk /usr/k/m1 [-r]
```

montează cu posibilitatea de citire (-r) la directorul gol m1 volumul /dev/disk.

Pentru a accesa o partiție aflată pe primul disc SATA, comanda de montare la directorul d1 din radacina este:

```shell
$ mount /dev/sda1 /d1
```

De exemplu, pentru a accesa sub Linux o partitie formatata DOS (fat,fat32) este necesara montarea prin comanda:

```shell
$ mount -t msdos /dev/sda1 /d1
```

Daca montarea nu mai este necesara si se dorește, eventual, o alta montare de pe

alt dispozitiv, se impune demontarea si eliberarea punctului de montare prin:

```
Shell
$umount /dev/sda1
```

4.2. Implementarea sistemului de fișiere

Kernelul lucrează la accesul fișierelor cu i-node-ul atașat.

Fiecare utilizator are asociată o tabela a fișierelor deschise pe baza careia se actualizează tabela fișierelor deschise din întregul sistem. Aceasta din urma permite gestionarea unei tabele de i-node-uri active cu ajutorul careia se identifica fișierele pe disc. Orice disc care contine un sistem de fișiere Unix are următoarea structura: \

blocul 0	blocul 1		2+isize	fsize		
Blocul de boot	Superbloc	Lista de inode-uri	Blocuri de date	Zona de swapping		

Fig 1. Dispunerea informațiilor pe un disc Unix.

Blocul 0 este rezervat unui incarcator. Blocul 1 conține **superblocul** care gestionează alocarea blocurilor. El conține următoarele informații:

- dimensiunea sistemului de fișiere;
- numărul blocurilor libere din sistemul de fișiere;
- lista blocurilor disponibile din sistemul de fișiere;
- indexul următorului bloc liber din lista blocurilor libere;
- dimensiunea listei de i-node-uri;
- numarul total de i-node-uri libere din sistemul de fișiere;
- lista i-node-urilor libere din sistemul de fișiere;
- indexul următorului i-nod liber din lista i-node-urilor libere;
- alte informații;
- fanion(flag) de modificare.

Superblocul este copiat în memoria internă. Nucleul actualizează periodic superblocul pentru a asigura corespondența reala cu sistemul de fisiere. Ca atare, se recomanda terminarea normala a unei sesiuni de lucru. Distrugerea superblocului cauzează imposibilitatea accesului la sistemul de fișiere.

Blocurile 2,3, ...1+isize sunt alocate pentru lista de i-node-uri. Fiecare i-node este de 64 octeți si descrie un fișier. I-node-ul 2 este folosit de directorul rădăcină (1 este rezervat pentru gestionarea blocurilor defecte).

Blocurile 2+isize pana la fsize-1 sunt utilizate pentru înregistrarea datelor. Toate fișierele și directoarele sunt păstrate aici. Dacă un fișier sau director consta din mai multe blocuri acestea nu trebuie sa fie contigue pe disc.

Zona de **swapping** servește pentru salvarea temporara a unor segmente de procese, care asteapta în memoria internă a calculatorului.

Pentru a deschide un fișier în directorul curent, sistemul citește fiecare intrare din director si compara numele fișierului cu numele de fișier memorat în fiecare intrare pana cand fișierul este găsit sau se conchide ca acesta nu exista. Dacă fișierul este prezent, sistemul extrage din intrare numărul de i-node, si îl folosește ca index in lista (tabela) i-node-urilor de pe disc, pentru a localiza i-node-ul si a-l aduce în memorie. I-node-ul este depus in tabela i-node-urilor din memorie, care contine toate i-node-urile fișierelor deschise. Tabela este gestionata de kernel.

Localizarea unui fișier precizat printr-o cale completa este un lucru puțin mai dificil. De exemplu, pentru calea `/usr/act/fis` sistemul localizeaza inițial directorul rădăcină (i-node-ul 2). Dupa aceasta cauta printre intrarile sale intrarea `usr` si găsește i-node-ul asociat ei. I-node-ul este adus în memorie si se determina blocul de pe disc care contine directorul `/usr`. Intrările acestui director sunt citite si comparate cu șirul `act`. Odată găsită intrarea, se extrage i-node-ul pentru directorul `/usr/act` si se citește blocul de pe disc referit de el. In final, se cauta șirul `fis` printre intrarile sale si se determina i-node-ul care ne conduce la blocul de date al fișierului. Se poate observa ca utilizarea căilor relative este mult mai convenabila nu numai pentru utilizator cat si pentru sistem, care astfel își reduce operațiile de căutare.

4.3. Drepturi de acces

Protectia accesului la fișiere într-un sistem cu mai mulți utilizatori este un aspect important. În Unix fiecare fișier conține în i-node-ul asociat un **identificator utilizator** (**UID**) si un **identificator de grup** (**GUID**) al proprietarului (vezi procese). Fiecare utilizator primește un număr specific, numit **număr de identificare** (**ID**). În momentul în care este creat un fișier el primește ca semn de recunoaștere ID-ul celui care la creat. Fiecare fișier conține în i-node-ul asociat trei seturi de cate trei biți. Seturile corespund proprietarului, grupului din care face parte proprietarul (grupul) si celorlalți utilizatori din sistem (alții). Pentru fiecare dintre aceștia exista dreptul de citire (Read), de scriere (Write) si de execuție (eXecute). Dreptul precizeaza unui proces operațiile (citire, scriere, execuție) pe care acesta le poate efectua asupra unui fișier. Pentru fișiere obișnuite semnificația drepturilor este evidentă. Pentru directoare dreptul de citire înseamnă drept de consultare (de afișare) a directorului (comanda ls). Dreptul de scriere înseamnă ca în acest director se pot crea noi fișiere, șterge fișiere, se poate monta un sistem de fișiere se pot adăuga sau șterge legături. Un director care are drept de execuție poate fi vizitat în timpul căutării unui fișier. Pentru fișiere speciale si pipe dreptul de citire scriere semnifica capacitatea de a executa apelurile sistem read sau write. Dreptul de execuție nu este important în acest caz.

Drepturile de acces ale unui fișier sunt pastrate intr-un cuvant de 16 biti cu urmatoarea semnificatie:

Bit(i)	**Semnificație**
0-3	Tipul fișierului.
4	Seteaza ID-ul utilizator în timpul execuției (suid).
5	Seteaza ID-ul grupului în timpul execuției (sgid).
6	Setare swapping (sticky bit). Fișierele care au acest bit pozitionat pe 1 rămân în zona de swapping. Un astfel de fișier este citit ulterior în întregime printr-o singura operatie (nu pe blocuri) în memorie.

7-9 Drept de citire, scriere, execuție pentru proprietar.

10-12 Drept de citire, scriere, execuție pentru grup.

13-15 Drept de citire, scriere, execuție pentru alții.

4.3.1. **Bitul sticky**

Dacă un fișier executabil are bitul sticky pozitionat, atunci la prima sa execuție o copie a zonei sale text este salvată în zona de swap (la terminarea procesului). Acest lucru permite programului sa se incarce în memorie mult mai repede, deoarece zona swap este gestionata ca o zona contiguă. Acest bit era folosit pentru programe de aplicație obișnuite, precum editoarele de text. Cu toate ca zona de swap era limitata tehnica a fost des utilizata. Actualmente sistemele Unix avand un sistem de memorie virtuala si un sistem de fisiere rapid nu mai folosesc aceasta tehnica.

Versiunile noi de Unix permit poziționarea bitului sticky pentru directoare. Dacă acest bit este pozitionat pentru director, un fisier din director poate fi șters sau redenumit dacă utilizatorul are drept de scriere in director si este proprietarul fișierului si directorului, sau este superuser.

4.3.2. **Bitii suid si sgid**. Identificatorul utilizator (UID) si identificatorul grupului de utilizatori (GIDU)

Fiecare utilizator primește un număr specific, numit identificator utilizator (UID). Acesta este un număr pozitiv asociat cu un nume utilizator de intrare în sesiune (login), aflat în fișierul ce conține baza de date de utilizatori /etc/passwd. La intrarea în sesiune a unui utilizator, comanda login face acest identificator UID al primului proces creat, interpretorul shell. Procesele descendente din shell mostenesc acest UID.

Utilizatorii sunt organizați în grupuri (a nu se confunda cu grup de procese), care au ID propriu numit identificator de grup (GIDU). Pentru un utilizator, acesta este preluat din fișierul de parole si făcut GIDU shell-ului propriu.

Grupurile sunt definite în fișierul de grup `/etc/group`. La intrarea în sesiune un utilizator poate sa creeze un alt grup. Acesta schimba GIDU procesului ce gestionează cererea (normal shell-ul, prin comanda newgrp), care apoi este mostenit de toți descendenții procesului.

Cei doi identificatori descriși se numesc UID real si GUID real, deoarece aceștia sunt reprezentativi pentru utilizatorul real (persoana care a deschis sesiunea). Fiecărui proces îi sunt asociați alți doi identificatori numiți UID si GUID efectiv. Aceștia sunt, de regula, identici cu cei reali corespunzători, dar pot fi si diferiți după cum vom vedea în continuare. ID efectivi se referă la drepturile de acces.

Fiecare fișier conține în i-node-ul sau un UID si un GUID al proprietarului. I-node-ul contine printre altele doi biți: **suid** si **sgid**.

Cand un proces este în execuție, sistemul de operare îi atribuie doi identificatori: UID real, egal cu cel al utilizatorului procesului (cel care la inițializat) si UID efectiv. Analog sistemul de operare creeaza doi identificatori relativ la grup.

În general, ID efectivi sunt identici cu ID reali, în afara cazului în care procesul executa un cod conținut într-un fișier asupra căruia nu are drepturi de execuție obișnuite, dar are bitul suid sau sgid pozitionat. În acest caz, procesul poate totuși să execute codul conținut în fișier, cu ID efectiv egal cu cel al proprietarului fișierului si diferit de cel real, cel al proprietarului de fișier, care a rămas nemodificat. Procesul are dreptul, cât timp executa codul în cauză, corespunzător proprietarului fișierului programului în curs. Dacă, de exemplu, acest proprietar este superuserul, procesul are temporar toate drepturile asupra sistemului. Dar singura acțiune care o poate realiza este cea definită în programul executat si probabil determinată de proprietarul fișierului, în acest caz superuserul. Îndată ce procesul executa codul unui program conținut într-un fișier neprotejat, UID efectiv (GUID efectiv), redevine egal cu cel real. Procesul continuă să se execute cu drepturile inițiale.

Uneori se dorește ca un utilizator comun să aibă, pentru un timp, privilegiile altui utilizator. Avand o configurație ca în Figura 2, pentru ca procesul P (care are alt proprietar) sa poată modifica datele din F, bitul **suid** al lui P trebuie pozitionat si utilizatorul U să poată lansa in executie pe P. În aceasta situatie în timpul execuției lui P, acesta avand bitul suid pozitionat, U are IDU efectiv același cu al lui P si astfel el poate modifica datele din F (accesul direct U la F nu e permis).

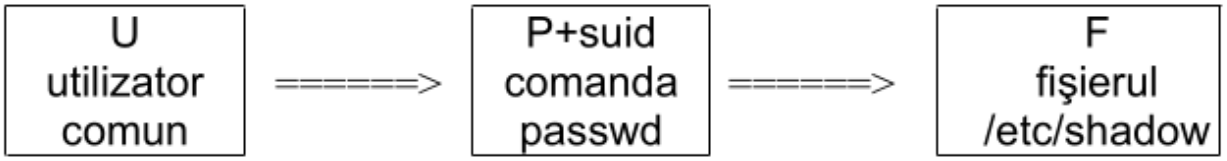

Fig.2. Accesul la fisierul de parole.

Deci utilizatorul poate avea acces la fisierul de parole numai prin intermediul unui program care are bitul **suid** pozitionat, iar acest program este astfel conceput încât un alt utilizator sa nu vadă sau sa modifice alte parole.

Bitul **sgid** este folosit în aceasi maniera pentru grup.

Un fisier care are acești biți pozitionati afișează ca rezultat al comenzii `ls` pe poziția lui 'x' litera 's'. Pentru a-i pozitiona se folosește comanda chmod. Proprietarul fișierului executabil și superuserul pot modifica acești biți. Se poate spune ca 's' este o extensie a permisiunii 'x' în contextul discutat.

Algoritmul folosit de sistem pentru a determina dacă un proces are sau nu dreptul de a efectua o operație (citire, scriere sau execuție) asupra unui fisier dat este următorul:

1) Dacă UID efectiv este 0 permisiunea este acceptata (utilizatorul efectiv este superuserul);
2) Dacă UID efectiv al procesului si UID al fișierului se potrivesc se decide permisiunea din biții proprietarului;
3) Dacă GUID efectiv al procesului si GUID al fișierului se potrivesc se decide permisiunea din biții grupului;
4) Dacă nici UID si nici GUID nu se potrivesc atunci, se decide din ultimul set de trei biți.

Sistemul pune la dispoziția utilizatorilor următoarele apeluri sistem pentru aflarea unui identificator real sau efectiv:

`int getuid()` Returneaza UID real.

`int getgid()` Returnează GUID real.

`int geteuid()` Returneaza UID efectiv.

`int getegid()` Returnează GUID efectiv.

Apelurile sistem pentru actualizarea UID si GUID sunt:

```
C/C++
int setuid(uid_t uid);
int setgid(gid_t gid);
```

Dacă apelantul este superuserul, aceste două apeluri sistem actualizează IDU real si efectiv sau GIDU la valoarea argumentului. Aceasta permite superuserului sa devina orice proces. Apelurile intorc **0** in caz de succes si **-1** in caz de eroare.

Cea mai importantă comanda Unix care folosește acest apel este `login`. După afișarea numelui de login si a parolei, programul login verifica acestea prin inspectarea fișierului `/etc/passwd` si dacă sunt valide execută `setuid` și `setgid` pentru a actualiza UID si GUID real și efectiv la valoarea din intrarea `/etc/passwd`. Login actualizează directorul de login ca director curent si foloseste apelul `exec` pentru lansarea shell-ului.

Utilizatorul simplu poate executa `setuid` și `setgid`, dar numai pentru a schimba UID efectiv sau GUID înapoi la ID real corespunzător. În acest caz, argumentul trebuie sa fie UID sau GUID real. Aceasta caracteristică este folosită cand un program trebuie lansat cu un ID efectiv particular, să facă o prelucrare privilegiata și să revină la ID real pentru restul duratei de viață ai procesului.

4.4. Comenzile `ls` si `chmod`

`ls`

Listeaza continutul directoarelor.

Sintaxa: `ls [optiuni] director`

Opțiuni:

-d Afișează numai directoarele din directorul curent.

-l Afișează în format lung, indicand drepturile de acces, numărul de legături, dimensiunea fișierului, data ultimei actualizari, numele fișierului.

-i Afiseaza si numarul i-node-ului fiecarui fișier.

-s Afișează numărul de blocuri pentru fiecare fișier.

-t Fișierele sunt sortate după data ultimei actualizari

-u La afișare se considera data ultimului acces în loc de data ultimei actualizari pentru opțiunile -t sau -l.

-r Inversează ordinea de sortare.

Exemplu: `ls -lsi /usr`

`chmod`

Schimba drepturile de acces la un fisier ordinar sau director.

Sintaxa: `chmod atr fisier(e)`

`atr` Se exprima ca un număr octal din patru cifre sau printr-o combinatie de forma:

`[u | g | o] [+ | - | =] [r | w | x | s | t]`

+ adaugă permisiune
- șterge permisiune
= atribuie permisiune

Categorie utilizator	Permisiune
u - proprietarul	r - Read
g - grupul	w - Write
o - restul utilizatorilor	x - eXecute
a - u+g+o	t - setare bit swapping

Drepturile de acces la un fișier se pastreaza într-un cuvant, plasat în i-node-ul fișierului. Se poate specifica direct valoarea acestui cuvânt, biți avand semnificatia:

Drept	Proprietar	Grup	Alţii
Citire	0400	040	04
Scriere	0200	020	02
Execuţie	0100	010	01

Exemplu:

`chmod o-x f1` Șterge dreptul la execuție pentru restul utilizatorilor fata de fișierul f1.

`chmod 0774 f1` f1 va avea permisiunile de acces `rwxrwxr--`.

5. Probleme rezolvate

5.1. Să se deruleze comenzile Linux/Unix corespondente următoarelor comenzi DOS (Command prompt in Windows) de lucru cu fișiere:

Comanda DOS	Comanda Linux/Unix
`attrib`	`chmod, ls -l`
`cd, chdir`	`cd, pwd`
`comp`	`cmp`
`copy`	`cp, cat`
`del, erase`	`rm`
`dir`	`ls`
`fc`	`cmp, diff`
`md, mkdir`	`mkdir`
`ren`	`mv`
`rd, rmdir`	`rmdir`
`type`	`cat`

5.2. Un fișier cu numele sistem.c este identificat în sistemul de operare DOS de expresiile 'sis*z.c*' si 'sistem.c??'. Care dintre aceste expresii identifica în Unix fișierul ?

Răspuns: Caracterul * identifica orice secvență arbitrară de caractere, iar caracterul ? identifica orice caracter singular. Diferențele apar la modul în care se face potrivirea. În DOS orice potrivire a unui nume de fișier cu o expresie ce conține caracterele * si ? este extinsă cu blancuri la opt caractere pentru nume si trei caractere pentru extensie.

În Unix, nici o expresie nu identifica fișierul. În prima există caracterul 'z' în plus, iar în a doua extensia ar trebui sa fie formată din trei caractere.

5.3. Se consideră că se dorește actualizarea unui fișier cu date furnizate de un alt utilizator care nu are permisiunea de acces direct la acel fișier. Situația aceasta este reprezentată schematic prin desenul de mai jos:

```
Programul lui------>Program ? -------->Datele lui
      X                                  Y
```

Care este calea prin care X va avea acces la datele lui Y ?

Răspuns: Calea prin care procesul X reușește sa faca actualizarea este printr-un program scris de Y ce rezidă într-un fișier, cu bitul suid pozitionat, inaccesibil utilizatorului dacă acesta nu este în execuție.

6. Probleme propuse pentru rezolvare

6.1. Comanda Unix *man* permite obținerea de informații despre comenzile Unix, diferite apeluri de sistem, utilitare importante. Comanda se apelează avand ca argument numele comenzii despre care se solicita informații. Utilizând comanda *man* obtineti informații suplimentare despre comenzile *ls*, *echo*, *cat* si *chmod*.

6.2. Ce realizeaza comenzile ?

```
ls -l ?                    ls [a-z]*[!0-9]
ls a*b                     ls *[!o]
ls -li                     ln unu doi
```

Sa se explice rezultatele obtinute.

6.3. Sa se parcurga arborele sistemului de fișiere și să se identifice directoarele cu comenzile, cu dispozitivele periferice, cu fișiere temporare, etc.

6.4. Să se vizualizeze conținutul unui director folosind comanda `ls -l`. Să se identifice informațiile din i-node. Utilizând comanda `chmod` sa se modifice drepturile de acces ale unui fișier executabil si ale unui director arbitrar din structura sistemului de fișiere. Sa se explice rezultatul comenzilor.

6.5. Ce se poate spune despre interpretarea caracterelor * si ? în Unix. Explicați diferențele.

6.6. Directorul /tmp este un exemplu în care bitul sticky poate fi utilizat. În acest director toata lumea are drepturi depline. În ce mod este util acest bit ?

6.7. Parola într-un sistem cu mai mulți utilizatori se schimbă des pentru siguranța datelor. Fiecare dintre utilizatori poate să-și schimbe singur parola folosind comanda *passwd*. Parola este păstrată într-un fișier comun tuturor utilizatorilor, care practic este modificat de fiecare utilizator care își schimba propria parola. Cu toate ca toți utilizatorii au acces la acest fișier ei pot să-și modifice doar parola proprie. Sa se explice cum este realizat acest lucru.

6.8. Din cele discutate în lucrare, s-ar părea că dacă se copie fișierul

/usr/bin/passwd în directorul propriu, devenim proprietarul copiei. Dacă se folosește comanda *chmod* se pozitioneaza bitul suid, iar prin comanda *chown* se modifica proprietarul la root. Executand acum copia vom obține privilegiul de a fi root. Acest lucru nu se produce, deci care este greșeală în raționament ?

7. Bibliografie

• Learn Linux, 101: A roadmap for LPIC-1
https://developer.ibm.com/tutorials/l-lpic1-map/
• CentOS Essentials
http://www.techotopia.com/index.php/CentOS_Essentials
• Linux Survival http://linuxsurvival.com/
• PC Computer boot:
• http://en.wikipedia.org/wiki/Bootstrapping_%28computing%29#Computing
• http://en.wikipedia.org/wiki/Booting#Boot_sequence_on_standard_PC_.28IBM-PC_compatible.29
• http://en.wikipedia.org/wiki/Master_boot_record
• X86 Assembly/Bootloaders
http://en.wikibooks.org/wiki/X86_Assembly/Bootloaders
• Master Boot Record (MBR)
https://en.wikipedia.org/wiki/Master_boot_record
http://ftp.kolibrios.org/users/Asper/docs/MBR/PartitionTables.txt
https://en.wikipedia.org/wiki/Booting#Boot_sequence
http://www.dewassoc.com/kbase/hard_drives/master_boot_record.htm
• GPT - GUID Partition Table http://en.wikipedia.org/wiki/GUID_Partition_Table
• http://linoxide.com/
http://linoxide.com/guide/linux-command-shelf.html
• OSDev.org
http://wiki.osdev.org/Main_Page
• Zed. A. Shaw, The Command Line Crash Course, http://cli.learncodethehardway.org/book/
• Vivek Gite et al., Linux Shell Scripting Tutorial, http://bash.cyberciti.biz/guide/Main_Page
• Filesystem Hierarchy Standard http://www.pathname.com/fhs/pub/fhs-2.3.pdf
• Linux initial RAM disk (initrd) overview https://developer.ibm.com/articles/l-initrd/
• Linux Boot Process Illustrated
https://www.linkedin.com/posts/alexxubyte_systemdesign-coding-interviewtips-activity-7322286227095752704-noW4/
• The Linux Boot Process
https://www.bglug.ca/articles/linux_boot_process.html
• The Linux boot process, a chart
http://sysadmin1138.net/mt/blog/2011/01/the-linux-boot-process-a-chart.shtml
• Systemd Boot Process a Close Look in Linux
http://linoxide.com/linux-how-to/systemd-boot-process/
• Ext4 Disk Layout
https://ext4.wiki.kernel.org/index.php/Ext4_Disk_Layout